TP1 - CPLP - Historia, evolución y características de Leng. de Programación

▼ Ejercicio 1

1951–1955: Lenguajes tipo assembly

Característica nueva: Programación de bajo nivel, muy cercana al hardware.

Lenguaje representativo: Assembly.

Se introduce una primera abstracción respecto al código máquina, permitiendo escribir instrucciones de forma simbólica.

1956–1960: FORTRAN, ALGOL 58, ALGOL 60, LISP

Característica nueva: Aparición de lenguajes de alto nivel.

Lenguajes:

- FORTRAN: orientado al cálculo científico.
- ALGOL 58/60: introduce estructuras de control y bloques, base de muchos lenguajes modernos.
- LISP: introduce la programación funcional basada en listas y recursión.

1961–1965: COBOL, ALGOL 60, SNOBOL, JOVIAL

Característica nueva: Diversificación de aplicaciones.

Lenguajes:

- COBOL: orientado a aplicaciones comerciales y manejo de datos.
- SNOBOL: especializado en procesamiento de cadenas de texto.
- JOVIAL: utilizado en sistemas militares.

1966–1970: APL, FORTRAN 66, BASIC, PL/I, SIMULA 67, ALGOL-W

Característica nueva: Aparición de nuevos paradigmas y simplificación del aprendizaje.

Lenguajes:

- BASIC: diseñado para ser fácil de aprender y usar.

- PL/I: lenguaje multipropósito.
- SIMULA 67: introduce la programación orientada a objetos.

1971–1975: Pascal, C, Scheme, Prolog

Característica nueva: Consolidación de paradigmas de programación.

Lenguajes:

- Pascal: orientado a la enseñanza de programación estructurada.
- C: combina eficiencia y control de bajo nivel, utilizado en sistemas.
- Scheme: programación funcional.
- Prolog: programación lógica.

1976–1980: Smalltalk, Ada, FORTRAN 77, ML

Característica nueva: Mejora en abstracción y seguridad.

Lenguajes:

- Smalltalk: programación orientada a objetos pura.
- Ada: fuerte tipado y confiabilidad.
- ML: lenguaje funcional con sistema de tipos fuerte.

1981–1985: Smalltalk 80, Turbo Pascal, Postscript

Característica nueva: Desarrollo de herramientas y entornos de programación más completos.

Lenguajes:

- Turbo Pascal: introduce entornos de desarrollo integrados (IDE).
- Smalltalk 80: consolida la programación orientada a objetos.

1986–1990: FORTRAN 90, C++, SML

Característica nueva: Integración de paradigmas.

Lenguajes:

- C++: combina programación estructurada y orientada a objetos.
- FORTRAN 90: incorpora mejoras modernas como manejo de arrays y modularidad.

1991–1995: TCL, PERL, HTML

Característica nueva: Programación para scripting y aparición de la web.

Lenguajes:

- PERL: automatización y procesamiento de texto.
- HTML: base para la creación de páginas web.

1996–2000: Java, JavaScript, XML

Característica nueva: Desarrollo de aplicaciones para la web y portabilidad.

Lenguajes:

- Java: portabilidad entre plataformas mediante la máquina virtual.
- JavaScript: programación del lado del cliente en navegadores.
- XML: formato estándar para intercambio de datos.

▼ Ejercicio 2

Lenguaje Python

Python nació a finales de la década de los 80s. Su autor principal, Guido van Rossum decidió empezar el proyecto como un pasatiempo dándole continuidad al lenguaje de programación ABC del que había formado parte del equipo de desarrollo en el CWI (Centrum Wiskunde & informática o Instituto Nacional de Investigación en Matemáticas e Informática).

El lenguaje de programación ABC se enfocaba en ser fácil de usar y aprender manteniendo potencia en su desempeño, este proyecto no trascendió como se esperaba debido al limitado hardware de la época.

El lenguaje de programación Python fue originalmente desarrollado para el sistema operativo Amoeba.

El nombre del lenguaje proviene del fanatismo de su creador con los humoristas Monty Python.

(podría extenderlo más, hay más info)

▼ Ejercicio 3

Un buen lenguaje de programación debería poseer una serie de atributos que faciliten su uso, comprensión y mantenimiento. Entre los más importantes se destacan:

- **Legibilidad:** El lenguaje debe ser fácil de leer y comprender, tanto para quien lo escribe como para otros programadores. Esto se logra mediante una sintaxis clara y consistente.

- **Simplicidad:** Debe contar con un conjunto reducido y coherente de reglas, evitando complejidad innecesaria. Cuanto más simple es el lenguaje, más fácil es aprenderlo y utilizarlo correctamente.
- **Ortogonalidad:** Significa que las características del lenguaje pueden combinarse de manera consistente y predecible, sin restricciones arbitrarias. Esto reduce la cantidad de excepciones y facilita el aprendizaje.
- **Expresividad:** Permite expresar ideas de forma clara y concisa, logrando resolver problemas con menos código y de manera más directa.
- **Abstracción:** Ofrece mecanismos para ocultar detalles innecesarios y manejar la complejidad, como funciones, clases o módulos.
- **Mantenibilidad:** Facilita la modificación y evolución del código a lo largo del tiempo, permitiendo corregir errores o agregar funcionalidades sin grandes dificultades.

Un ejemplo de lenguaje que cumple con estas características es **Java**, ya que presenta una sintaxis clara (legibilidad), una estructura bien definida (simplicidad), un modelo consistente de objetos (ortogonalidad), permite expresar soluciones de manera organizada (expresividad) y ofrece herramientas como clases y paquetes para manejar la complejidad (abstracción).

▼ Ejercicio 4

Se elige el lenguaje **Java**.

- **Tipos de expresiones**

En Java se pueden escribir distintos tipos de expresiones, es decir, construcciones que al evaluarse producen un valor. Entre ellas se destacan:

- **Expresiones aritméticas:** realizan operaciones matemáticas, por ejemplo a + b, x * y.
- **Expresiones relacionales:** comparan valores y devuelven un booleano, por ejemplo a > b, x == y.
- **Expresiones lógicas:** combinan condiciones booleanas, por ejemplo a && b, !existe().
- **Expresiones de asignación:** asignan valores a variables y también devuelven un resultado, por ejemplo a += 10.

- **Expresiones de invocación de métodos:** ejecutan métodos y devuelven su resultado, por ejemplo sumar(a, b) o obj.getNombre().
- **Facilidades para la organización del programa**

Java ofrece diversas herramientas para estructurar y organizar el código:

- **Clases:** permiten agrupar datos (atributos) y comportamiento (métodos) en una misma unidad.
- **Métodos:** posibilitan dividir la lógica en partes más pequeñas y reutilizables.
- **Paquetes:** permiten organizar las clases en módulos, facilitando la separación de responsabilidades.
- **Encapsulamiento:** permite ocultar los detalles internos de una clase y exponer solo lo necesario.
- **Herencia y polimorfismo:** facilitan la reutilización de código y permiten escribir programas más flexibles y extensibles.

▼ Ejercicio 5

Ada es un lenguaje fuertemente tipado y muy estricto con los tipos de datos. Realiza chequeos en tiempo de compilación para evitar errores y no permite conversiones implícitas peligrosas. Posee tipos básicos como enteros, reales, booleanos y caracteres, y también permite definir nuevos tipos mediante subtipos, tipos enumerados y tipos derivados, lo que mejora la seguridad y claridad del programa.

Los tipos abstractos de datos en Ada se implementan mediante paquetes. Un paquete se divide en una especificación, que define la interfaz pública, y un cuerpo, que contiene la implementación. Esto permite encapsular datos y ocultar información, separando claramente qué se puede usar desde afuera y cómo está implementado internamente.

En cuanto a estructuras de datos, Ada soporta arreglos, registros y punteros (llamados access). Los arreglos permiten almacenar colecciones de elementos, los registros agrupan distintos tipos de datos bajo una misma estructura, y los access permiten manejar estructuras dinámicas. Todo esto se maneja bajo un sistema de tipos estricto.

El manejo de excepciones en Ada está integrado en el lenguaje mediante bloques exception. Permite capturar y tratar errores en tiempo de ejecución,

como errores de restricciones, de programa o de memoria, lo que contribuye a la robustez del software.

Por último, Ada tiene soporte nativo para concurrencia mediante tareas (tasks), que funcionan de forma similar a hilos. Las tareas pueden ejecutarse en paralelo y comunicarse entre sí mediante mecanismos de sincronización como el rendezvous. Esto permite desarrollar sistemas concurrentes de manera segura, evitando problemas como condiciones de carrera.

▼ Ejercicio 6

Java fue creado como una herramienta de programación para ser usada en un proyecto de set-top-box en una pequeña operación denominada The Green Proyect en Sun Microsystems en 1991. El equipo (*green team*), compuesto por trece personas y dirigido por James Gosling, trabajó durante 18 meses en Sand Hill Road, Menlo Park (California), para desarrollarlo.

Java introdujo varios cambios importantes en la Web, especialmente en sus primeras etapas (años 90 y principios de los 2000). Los principales fueron:

- **Contenido dinámico del lado del cliente (Applets)**
  Antes de Java, las páginas web eran mayormente estáticas. Con los *applets*, se podían ejecutar programas dentro del navegador, permitiendo animaciones, juegos y aplicaciones interactivas directamente en la web.
- **Interactividad avanzada**
  Java permitió interfaces más ricas que HTML puro, como formularios dinámicos, gráficos interactivos y validaciones más complejas en el cliente.
- **Portabilidad (Write Once, Run Anywhere)**
  Gracias a la máquina virtual de Java (JVM), el mismo programa podía ejecutarse en distintos sistemas operativos sin modificaciones, lo cual fue clave para la web multiplataforma.
- **Programación del lado del servidor**
  Java también revolucionó el backend con tecnologías como:
  - **Servlets**: generación dinámica de contenido web.
  - **JSP (JavaServer Pages)**: mezcla de HTML con código Java.
    Esto permitió crear sitios web dinámicos (como los actuales), conectados a bases de datos.

- **Mayor seguridad**
  Los applets se ejecutaban en un entorno controlado (*sandbox*), lo que reducía riesgos de seguridad al correr código en el navegador.
- **Base para frameworks empresariales**
  Java sentó las bases para el desarrollo de aplicaciones web empresariales robustas (ej: Spring, aunque vino después), con arquitectura en capas y alta escalabilidad.

En resumen: Java transformó la web de un sistema estático de documentos a una plataforma capaz de ejecutar aplicaciones completas, tanto en el cliente (en su momento) como en el servidor (hasta hoy).

Java es un lenguaje independiente de la plataforma donde se ejecuta, esto significa que los programas escritos en este lenguaje pueden ejecutarse igualmente en cualquier tipo de hardware.

Para lograr esto, se compila el código fuente escrito en lenguaje Java, para generar un código en bytecode (especialmente Java Bytecode), instrucciones máquina simplificadas específicas de la plataforma Java. El bytecode es ejecutado entonces en la máquina virtual (JVM), un programa escrito en código nativo de la plataforma destino (que es el que entiende su hardware), que interpreta y ejecuta el código. Además, se suministran bibliotecas adicionales para acceder a las características de cada dispositivo (como los gráficos, ejecución mediante hebras o threads, la interfaz de red) de forma unificada. Se debe tener presente que, aunque hay una etapa explícita de compilación, el bytecode generado es interpretado o convertido a instrucciones máquina del código nativo por el compilador JIT (Just In Time).

Lenguaje Java → Java bytecode → JVM + JIT

▼ Ejercicio 7

Java esta basado sobre los lenguajes C y C++ adoptando su sintaxis familiar de estilo llaves, comentarios y estructura, pero simplificando la gestión de memoria. Java tiene menos utilidades de bajo nivel que cualquiera de ellos.

El lenguaje se denominó inicialmente *Oak* (por un roble que había fuera de la oficina de Gosling), luego pasó a llamarse *green,* tras descubrir que *Oak* era ya una marca comercial registrada para adaptadores de tarjetas gráficas, y finalmente se le renombró *Java.*

▼ Ejercicio 8

Applets

Los applets son programas incrustados en otras aplicaciones, normalmente una página web que se muestra en un navegador. Cuando un navegador carga una página web que contiene un *applet*, este se descarga en el navegador web y comienza a ejecutarse utilizando la Java Virtual Machine (JVM). Esto permite crear programas que cualquier usuario puede ejecutar con tan solo cargar la página web en su navegador.

Ventajas:

- Son multiplataforma.
- El mismo applet puede trabajar en todas las versiones de Java.
- Son compatibles con la mayoría de navegadores.
- Pueden ser almacenados en la memoria cache.
- Puede ejecutarse a velocidades comparables a las de otros lenguajes compilados, como C++.

Desventajas:

- Requiere el plugin de Java.
- No puede iniciar la ejecución hasta que la JVM este en funcionamiento.
- Un applet podría exigir una versión especifica del JRE (Java Runtime Enviroment).
- Puede tener vulnerabilidades que permitan ejecutar código malicioso.
- Si no está firmado como confiable, tiene un acceso limitado al sistema del usuario.

Servlets

Un servlet es una clase en el lenguaje de programación Java, utilizada para ampliar las capacidades de un servidor, son los encargados de generar respuestas a las peticiones recibidas de los clientes.

Aunque los servlets pueden responder a cualquier tipo de solicitudes, estos son utilizados comúnmente para extender las aplicaciones alojadas por servidores web, de tal manera que pueden ser vistos como applets que se ejecutan en servidores en vez de navegadores web.

▼ Ejercicio 9

La estructura del lenguaje C se basa en un conjunto de funciones, por lo que se denomina un “lenguaje procedimental”. El programa siempre comenzará por la ejecución de la función **main**. Cada función debe contener:

- Una cabecera de la función, que consta del nombre de la función, seguido de una lista opcional de argumentos encerrados con paréntesis.
- Una lista de declaración de argumentos, si se incluyen estos en la cabecera.
- Una sentencia compuesta, que contiene el resto de la función.

Cada sentencia compuesta se encierra con un par de llaves, { . . . }. Las llaves pueden contener combinaciones de sentencias elementales (denominadas s*entencias de expresión*) y otras sentencias compuestas. **Así las sentencias compuestas pueden estar anidadas**, una dentro de otra. Cada sentencia de expresión debe acabar en punto y coma (;).

Ejemplo:

```
#include <stdio.h>

// Cabecera de función con argumento
int sumar(int a, int b) {
    // Sentencia compuesta (bloque)
    int resultado;          // declaración

    resultado = a + b;      // sentencia de expresión

    return resultado;       // sentencia de expresión
}

// Función principal
int main() {
    int x = 5;              // declaración + inicializac
ión
    int y = 3;
    int res;
```

```
    res = sumar(x, y);      // llamada a función

    // Sentencia compuesta anidada
    if (res > 0) {
        printf("Resultado positivo: %d\n", res);
    }

    return 0;
}
```

C no tiene funciones anidadas porque:

- prioriza **simplicidad, eficiencia y control de memoria**
- no implementa **closures ni acceso a variables externas**
- mantiene un modelo de ejecución basado en funciones independientes

▼ Ejercicio 10

El lenguaje C maneja las expresiones de forma flexible y eficiente, permitiendo combinar constantes, variables, operadores y llamadas a funciones para obtener un valor.

Una expresión es cualquier combinación válida de estos elementos que produce un resultado. En C existen distintos tipos de expresiones: aritméticas (para realizar cálculos), relacionales (para comparar valores), lógicas (para combinar condiciones) y de asignación (para asignar valores a variables).

El lenguaje utiliza reglas de precedencia y asociatividad para determinar el orden en que se evalúan los operadores dentro de una expresión, pudiendo modificarse este orden mediante el uso de paréntesis.

Además, C realiza conversiones de tipo implícitas cuando es necesario, adaptando los operandos a un tipo común para poder operar.

En las expresiones lógicas, C aplica evaluación de corto circuito, lo que significa que la evaluación se detiene cuando el resultado ya puede determinarse.

Finalmente, una característica importante es que las expresiones pueden constituir sentencias completas cuando finalizan con punto y coma.

▼ Ejercicio 11

Los lenguajes **Python, Ruby y PHP** permiten desarrollar principalmente aplicaciones del lado del servidor (backend), aunque cada uno también se utiliza en otros tipos de programas según sus características.

En el caso de **Python**, se pueden desarrollar aplicaciones web, programas de automatización, software científico, inteligencia artificial, análisis de datos y scripts en general. Es un lenguaje **multiparadigma**, ya que soporta programación orientada a objetos, programación imperativa y también elementos de programación funcional.

Ruby se utiliza principalmente para el desarrollo de aplicaciones web, especialmente mediante el framework Ruby on Rails. También permite la creación de scripts y herramientas de automatización. Es un lenguaje **orientado a objetos puro**, donde prácticamente todo es un objeto, aunque también admite características de otros paradigmas.

Por su parte, **PHP** está orientado principalmente al desarrollo de aplicaciones web del lado del servidor, como sitios dinámicos, sistemas de gestión de contenido (CMS) y APIs. Es un lenguaje **imperativo y orientado a objetos**, ya que combina programación estructurada con soporte para clases y objetos.

En cuanto a los paradigmas:

- La **programación imperativa** se basa en instrucciones que modifican el estado del programa mediante secuencias de comandos.
- La **programación orientada a objetos** organiza el código en objetos que combinan datos y comportamiento, utilizando conceptos como clases, herencia, encapsulamiento y polimorfismo.
- La **programación funcional** se basa en el uso de funciones como elementos principales, evitando efectos secundarios y promoviendo la inmutabilidad.

Las características que determinan la pertenencia a cada paradigma incluyen la forma en que se estructura el código, el manejo del estado, el uso de funciones o clases, y las abstracciones que el lenguaje ofrece para resolver problemas.

▼ Ejercicio 12

Además de sus usos y paradigmas, los lenguajes **Python, Ruby, PHP, Gobstone y Processing** presentan otras características importantes

relacionadas con el tipado de datos, la organización de los programas y su forma de ejecución.

Python es un lenguaje de **tipado dinámico y fuerte**, lo que significa que no es necesario declarar el tipo de las variables, pero sí se controla que las operaciones sean válidas entre tipos. Los programas se organizan en módulos y paquetes, y utiliza la **indentación como parte de la sintaxis** para definir bloques de código. Es interpretado y tiene una sintaxis clara y legible.

Ruby también posee **tipado dinámico y fuerte**, y se caracteriza por ser completamente **orientado a objetos**, donde todo es un objeto. Los programas se estructuran en clases y módulos. Su sintaxis es flexible y expresiva, permitiendo múltiples formas de escribir lo mismo. Es interpretado.

PHP tiene **tipado dinámico y débil** (aunque en versiones modernas puede ser más estricto si se configura). Se utiliza principalmente embebido dentro de HTML, lo que influye en cómo se organizan los programas. Permite programación estructurada y orientada a objetos. Es interpretado en el servidor web.

Gobstone es un lenguaje educativo con **tipado estático y fuerte**, diseñado para enseñar conceptos de programación. Los programas se organizan en procedimientos y funciones, y trabajan sobre un “tablero” con reglas bien definidas. Tiene una sintaxis simple y estricta que facilita el aprendizaje de la lógica y el control de flujo.

Processing es un lenguaje basado en Java con **tipado estático**, orientado a la programación visual y artística. Los programas se organizan en funciones principales como `setup()` y `draw()`, que controlan la inicialización y el ciclo de ejecución. Es muy utilizado para gráficos, animaciones e interacción visual.

En general, estas características (tipo de tipado, organización del código, forma de ejecución y nivel de abstracción) influyen en cómo se desarrollan los programas y en los problemas para los cuales cada lenguaje resulta más adecuado.

▼ Ejercicio 13

JavaScript es un lenguaje que pertenece al paradigma **multiparadigma**, ya que permite programar utilizando distintos enfoques:

- **Imperativo**, mediante secuencias de instrucciones.

- **Orientado a objetos**, basado en prototipos (no en clases tradicionales, aunque hoy las soporta).
- **Funcional**, ya que las funciones son de primera clase y se pueden pasar como parámetros o retornar.

En cuanto al **tipo de lenguaje**, JavaScript es un lenguaje de **alto nivel**, **interpretado** (aunque actualmente también es compilado JIT por los motores modernos) y de **tipado dinámico y débil**.

Se utiliza principalmente para el desarrollo web, tanto en el **frontend** (en navegadores) como en el **backend** (por ejemplo, con Node.js).

▼ Ejercicio 14

JavaScript presenta diversas características importantes relacionadas con el manejo de datos, variables y control de errores.

En cuanto al **tipado de datos**, es un lenguaje de **tipado dinámico y débil**, lo que significa que las variables no requieren declarar su tipo y pueden cambiarlo durante la ejecución. Además, realiza conversiones automáticas de tipo (coerción).

Respecto a las **variables**, se pueden declarar utilizando `var`, `let` y `const`.

- `var` tiene alcance de función.
- `let` y `const` tienen alcance de bloque.
- `const` se utiliza para valores que no deben reasignarse.

En el manejo de **excepciones**, JavaScript utiliza las estructuras `try`, `catch` y `finally`, lo que permite capturar y manejar errores en tiempo de ejecución. También se puede generar errores de forma manual con la instrucción `throw`.

En cuanto a la **organización del código**, JavaScript permite estructurar programas mediante funciones, objetos y módulos. Las funciones son de primera clase, lo que significa que pueden asignarse a variables, pasarse como argumento y retornarse desde otras funciones.

Además, es un lenguaje **basado en prototipos**, donde la herencia se implementa a través de prototipos en lugar de clases tradicionales (aunque hoy existen clases como sintaxis moderna).

Por último, JavaScript es un lenguaje **event-driven y asíncrono**, lo que permite manejar eventos y ejecutar operaciones no bloqueantes mediante callbacks, promesas y `async/await`.

▼ Bibliografía

Historia de Python - Wikipedia, la enciclopedia libre

Python - Wikipedia, la enciclopedia libre

Java (lenguaje de programación) - Wikipedia, la enciclopedia libre

Applet Java - Wikipedia, la enciclopedia libre

Java Servlet - Wikipedia, la enciclopedia libre

Capítulo 1. Estructura de un programa en C

estructura de un programa en C

¿Qué es Python y para qué se usa? Guía para principiantes | Coursera

PHP - Wikipedia, la enciclopedia libre

JavaScript - Wikipedia, la enciclopedia libre

Processing - Wikipedia, la enciclopedia libre

Ruby - Wikipedia, la enciclopedia libre

Gobstones - Wikipedia, la enciclopedia libre